OPENSTUDY · AI 学习智能体

把不会的，问到会

会读你资料的 AI 学习平台

把任意学习材料，编译成可执行、可追溯、会进化的学习资产包

对话是入口，资产是结果。

一句话完成：体检 / 整理 / 出题 / 微课 / 装配 / 导出

华为云码道 CodeArts 开发 · GitCode 开源 · FastAPI + Vue 3

gitcode.com/qi142410131/openstudy-fastapi-ts



真实运行界面 · 对话即入口

先体检，再整理 **看得见的手术方案** **拖拽装配资产**

学生不缺资料，缺的是「把资料编译成学习资产」

资料在硬盘里，学习在对话外：没有一条从「混乱材料」到「可执行资产」的稳定通路。

01

资料乱、散、重复

Word 里夹着广告图、重复段落、页码页眉；人工整理一份要半小时，还不一定舍得删。

乱

02

一份资料只用一次

图片、公式、表格散落在文档里；想变成题库、讲义、复习卡、微课，只能重新找、重新截。

断

03

AI 生成是黑盒

改了什么、为什么改、改错怎么办——通用 AI 只给结果，不给可审阅的过程。

黑

用户原话 **“我只想把我自己的资料，变成能刷的题、能讲的课、能打印的讲义。”**

可执行

题目、讲义、复习卡、微课，拿到就能直接学

可追溯

每道题、每条结论，都能回到原文的那一句

会进化

每次练习与报告，都会让这份资产变新

先体检，再整理：每份资料都有诊断书

上传后第一屏不是文件列表，而是四维体检：结构、图文、知识覆盖、可学习化。



体检报告里有什么

结构完整度 81
识别到 8 个标题、6 个内容区块

图文相关度 100
3 张图都能定位到题目/章节

知识覆盖度 100
5 个章节都有完整讲解

可学习化 36
0 个章节有例题或练习

问题清单

P1 所有章节都没有例题或练习

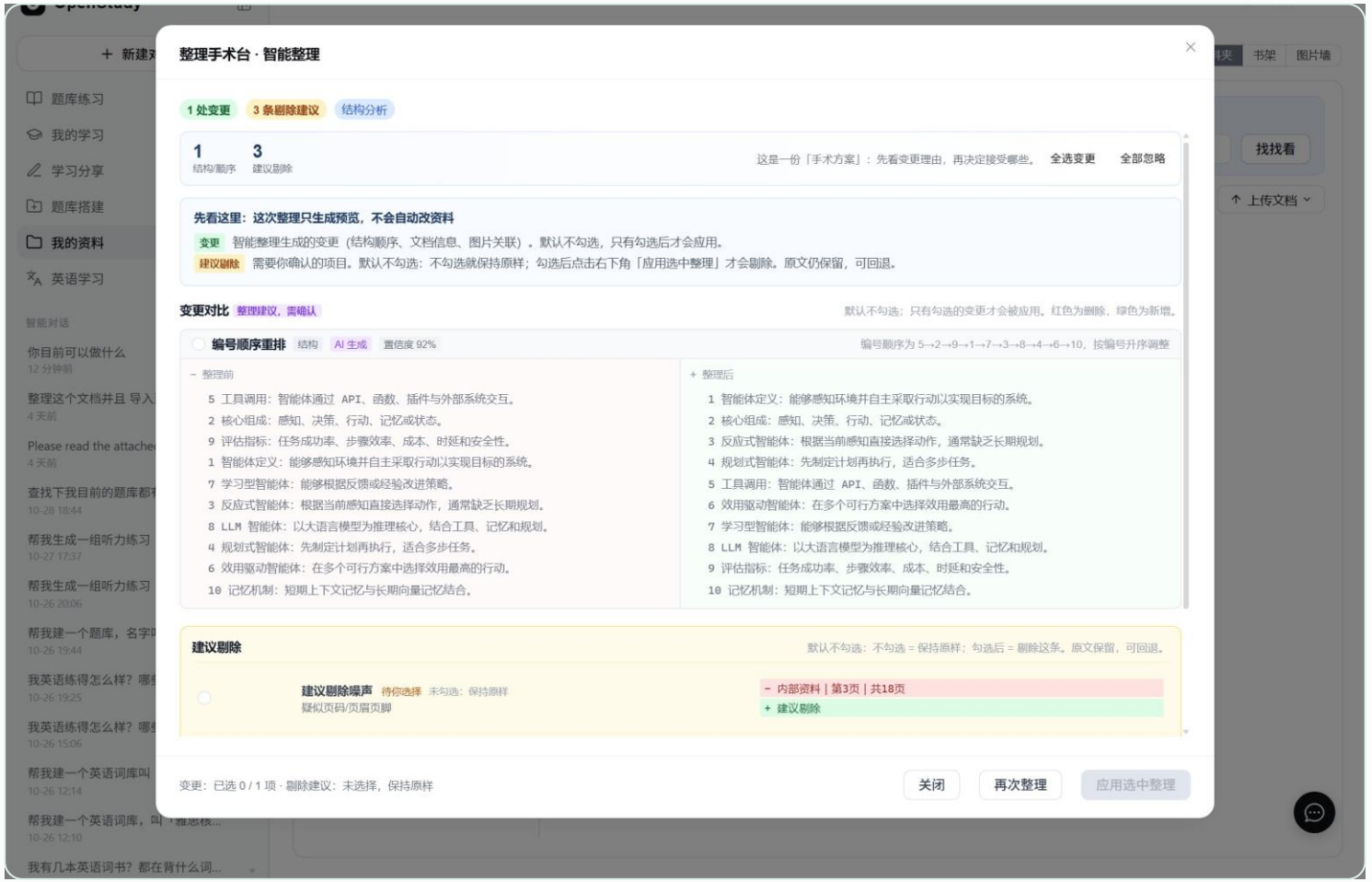
建议：按章节生成靶向题，变成可练资产

体检	手术台	复检
先看诊断	逐项确认	再看变化

真实运行界面：资料体检 · 四维评分 / 问题清单 / 章节热力

AI 改了什么，一条条摊开给你看

不是一键黑盒：删除、合并、重排、改写都有原因和置信度，可逐项接受、拒绝、回退。



手术方案

1	3	0	0
结构 / 顺序	建议剔除	图片	元信息

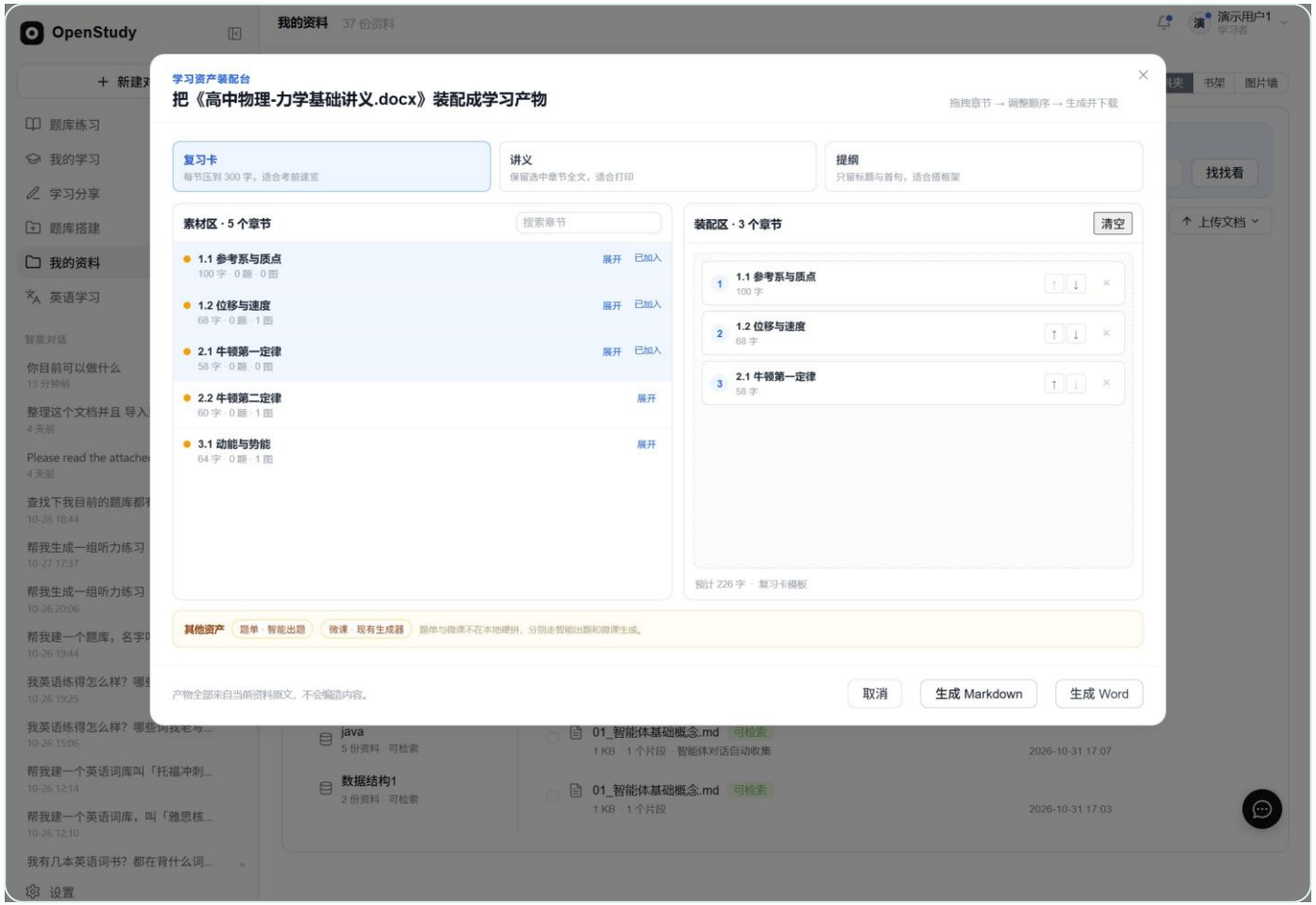
- 左原文 / 右整理后，点一条两边同步高亮
- 每条变更都有理由 + 置信度，不是“AI 觉得”
- 可全选、可逐条拒绝、可一键回退
- 应用后点「复检」，回到同一份体检报告

把“黑盒自动化”变成
“可解释、可干预、可回滚的白盒协作”

真实运行界面：变更对比 · 逐项勾选 · 建议剔除

拖拽章节，装配成不同的学习产物

对话负责生成，装配台负责把结果变成能带走、能打印、能复习的资产。



可装配的产物

讲义　复习卡　提纲

题单　微课

01　拖章节
左侧素材拖进装配区

02　调顺序
装配区内拖拽排序

03　出产物
Markdown / Word 直接带走

“用这份资料做一份复习卡，重点第 2 章。”
一切皆对话：说一句，Agent 去读、去建、去生成

真实运行界面：素材区拖拽 → 装配区排序 → 生成 Markdown / Word

Word 题库导入：图、公式、解析都不丢

最常见的一份资料：Word 题库，题干里有复杂公式，后面还跟着图。

Word 原文

一、极限与连续

（1）计算极限：

$$\lim_{x\to 0}\frac{e^x-1-x}{x^2}$$

二、偏导数与全微分

（2）设 z = arctan(y/x)，求：

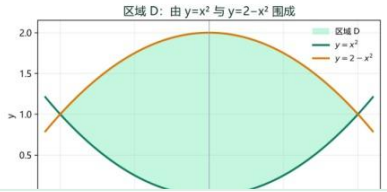
$$\frac{\partial^2 z}{\partial x\partial y}$$

三、二重积分

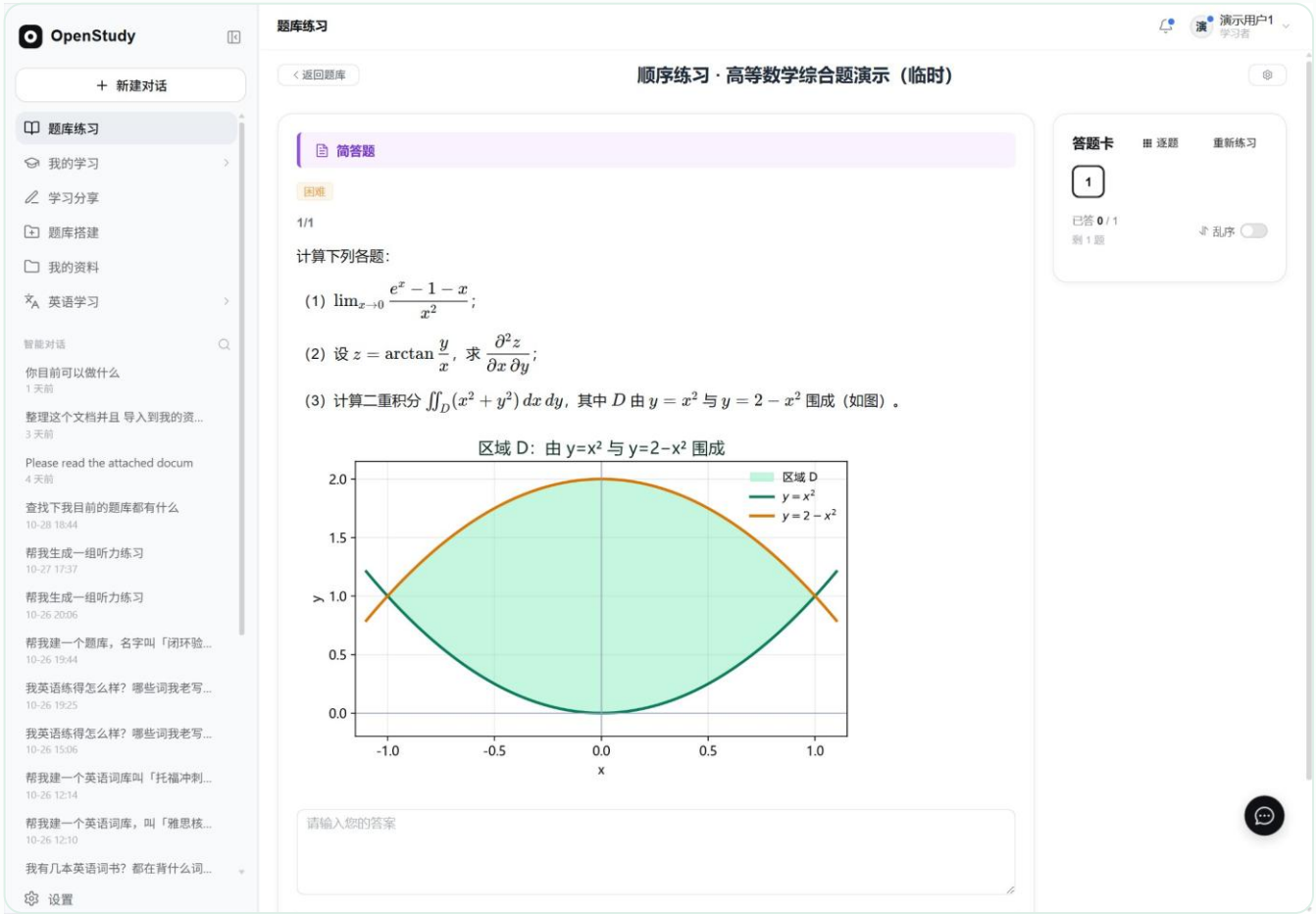
（3）计算二重积分：

$$\iint_D (x^2+y^2)dxdy$$

其中 D 由 y = x² 与 y = 2 − x² 围成（如图）。



导入后的题目详情



- 题干 / 选项 / 答案自动拆开
- 图片按原位置归属到题目
- 复杂公式渲染成 LaTeX / KaTeX

Agent 系统架构：一个会话，多个专家

路由不花 LLM，写操作可确认，文件和图片只上传一次——这是我们的 Agent 设计。

入口层

对话 | 附件 / 图片 | 我的资料 | 题库 | 微课 | 分享

编排层

Orchestrator
关键词路由 · 零 LLM

→

5 类子代理
题库 · 资料 · 笔记 · 统计 · 智能体（可并行）

→

20+ 工具
统一回执 · 可确认 · 可 undo

支撑层

记忆与文件复用
会话记忆 / 资料记忆 / 长期记忆

确认与回退
confirm_ticket / 四域整树快照

流式与结果卡
SSE 逐字 / 工具结果落地

RAG 与数据
解析 / 向量 / 混合检索

一次真实对话：同一张图，只上传一次

不用重复上传

01 **用户上传图片**
附件进入会话

02 **会话记住 fileId**
持久化，不丢图

03 **下一轮说**
“把刚才那张图插到第 4 题”

04 **chat_file_manage**
按 fileId 取回原图

05 **question_manage**
写入解析，结果卡确认

华为云 · 码道 CodeArts

码道不是补全插件，是一条完整的 AI 编码流水线

从需求到沉淀，8 个环节分别对应一项码道能力，每一步都有可核对的产物。

① 需求	② 接入	③ 读码	④ 写测	⑤ 审查	⑥ 修复	⑦ 提交	⑧ 沉淀
用户目标 · 一句话	git status · 分支	语义检索	developer-test-agent	diff · security_audit	/bugfix 修复专家	git commit / push	experience · export

支撑能力：规则 AGENTS.md • 技能 SKILL.md • 经验 experience • 沙箱 / 权限 • checkpoint / undo

真实案例：为 html_sanitize 补测试 → 测出 data: 协议漏检 → /bugfix 修复 → 回归通过后提交。

开发过程可核对：会话可导出 • 技能可复用 • 提交可追溯

过程记录统一放在 doc/dev-process/，可现场按会话 ID 抽查。

码道把 AI 编码从「补全」变成「可核对的过程」。

用码道的哪个能力，开发了哪块系统

不是功能清单，是实际开发中的分工：每张卡都是「功能 → 模块 → 怎么用 → 结果」。

子代理 • 测试

题库与考试链路

developer-test-agent 生成 58 条单测，覆盖两条清洗路径。

→ 58 passed

修复专家

智能体工具链

/bugfix 按 bug 单定位并修复降级正则，回归通过后才提交。

→ P1 修复

语义检索 • 索引

Agent Runtime / RAG

CodeSemanticSearch + 代码库索引，先读后写，跨全库定位调用点。

→ file:line

规则 • 技能

团队协作规范

AGENTS.md 写规则、SKILL.md 写流程，随仓库分发，换人也能照着做。

→ 8 步流程

会话

长任务调试

/fork 从任意消息分叉，/timeline 回跳，/export 导出。

→ 可追溯

GitCode • git

提交与交付

分支开发、中文提交、推送远端，一次完成。

→ 596959b

一张卡就是一条链路：码道功能 → 开发的系统模块 → 怎么用 → 可核对结果。

09 / 现场演示与下一步

3 分钟看闭环，下一步把「好用」做成规模

先让观众看到资料真的变成了资产，再交代我们接下来补什么、谁来做。

0:00 – 0:30

资料体检

上传一份混乱 Word

四维评分、P1 问题、章节热力图

0:30 – 1:20

整理手术台

一键整理，生成 3 条建议

逐条看理由；接受 2 条、拒绝 1 条；复检

1:20 – 2:10

资产装配

拖 3 个章节进装配区

生成复习卡 / 讲义；题单走智能出题

2:10 – 3:00

学习闭环

做错一道题，打开诊断

点「原文依据」回原句；生成 3 道变式题

下一步

30 天 知识覆盖热力图升级 · 多模态向量 · 移动端适配

90 天 教师端 / 多租户 · OBS 直传 · 题库与资料开放 API

持续 把每条学习结论都做成可追溯、可复检的资产

团队分工

产品 / 后端 场景定义、Agent 与 RAG 后端、数据与部署

前端 / 交互 对话体验、资料空间、题库与微课面板

测试 / 演示 探针与回归、演示脚本、材料与讲解

谢谢。欢迎现场提问，也欢迎直接上手体验。

gitcode.com/qi142410131/openstudy-fastapi-ts